

Capstone 4 — Domain A: End-to-End Pre-Auth Pipeline

Build a 4-agent pipeline that processes pre-authorization requests end-to-end: intake validation, clinical criteria evaluation, decision with human-in-the-loop, and communication — with circuit breaker protection.

Project Brief

BUSINESS CONTEXT

A health plan processes 500+ prior authorization requests daily. In Capstone 3, you built a single ReAct agent that reasons through one request. But a single agent handling the entire lifecycle — intake validation, clinical evaluation, decision-making, and provider communication — creates a monolithic bottleneck. When the decision logic changes, you redeploy everything. When intake parsing fails, clinical evaluation stops. When you want to A/B test notification templates, you risk breaking the decision engine.

The production solution is a **pipeline of specialized agents**. Each agent has a focused responsibility, its own tools, and a typed interface for passing structured state to the next agent. The Intake Agent validates; the Clinical Criteria Agent evaluates; the Decision Agent determines (or pauses for human review); the Communication Agent drafts and sends. Each can be independently deployed, tested, and scaled.

Three production patterns make this pipeline trustworthy: human-in-the-loop checkpoints for medium-confidence decisions, a circuit breaker that halts processing when errors cascade, and structured logging at every agent transition for full observability.

WHAT YOU WILL BUILD

A 4-agent pipeline with HITL and circuit breaker:

- **Agent 1 — Intake:** Validates request schema, verifies member eligibility, confirms provider status. Input guardrails: PII detection, schema validation.
- **Agent 2 — Clinical Criteria:** Fetches policy criteria, evaluates each criterion against submitted evidence, generates per-criterion confidence scores.
- **Agent 3 — Decision:** Computes overall confidence. If >90%: auto-decide. If 70–90%: route to human reviewer. If <70%: auto-deny with appeal instructions.
- **Agent 4 — Communication:** Drafts determination letter (approval/denial/info-request), validates HIPAA compliance, sends via appropriate channel.

Skills practiced: Multi-agent architecture (M14), HITL patterns (M17), input/output guardrails (M16–M17), circuit breaker, evaluation (M18), structured inter-agent state.

Prerequisites

Complete these modules before starting this capstone:

- **M03–M05 — Tool Use Fundamentals:** Each agent in this pipeline uses tool calling to invoke mock healthcare APIs. You need to be comfortable defining tool schemas and handling tool results.
- **M09 — RAG (Retrieval-Augmented Generation):** Agent 2 (Clinical Criteria) performs policy document lookup — a simplified RAG pattern where clinical criteria are fetched and matched against submitted evidence.
- **M12 — ReAct Pattern:** Each agent internally uses the observe → think → act loop. The Clinical Criteria Agent reasons through each criterion step by step.
- **M14 — Multi-Agent Systems:** This capstone is a 4-agent pipeline with typed state passing between agents. You need to understand coordinator patterns and inter-agent contracts.
- **M16 — Input Guardrails:** The Intake Agent applies input validation (schema checks, missing field detection) before any processing begins. You need to understand how guardrails prevent garbage-in-garbage-out in agent pipelines.

- **M17 — HITL & Output Guardrails:** The Decision Agent routes medium-confidence cases to a human clinical reviewer. Output guardrails (HIPAA compliance checks on determination letters) are applied at the Communication Agent boundary.

You should also be comfortable with Python classes, JSON data structures, and running scripts from the command line.

Environment Setup

WHAT YOU WILL BUILD

A 4-agent pre-authorization pipeline with human-in-the-loop review for medium-confidence clinical decisions, input/output guardrails, and a circuit breaker for cascading failure protection.

Time estimate: 2–3 hours Difficulty: ★★★★★☆

Requirements

- Python 3.10 or higher
- An Anthropic API key (get one at console.anthropic.com)

Install Everything

Copy and paste this entire block into your terminal:

UNIX / MACOS · WINDOWS

UNIX / MACOS

```
mkdir capstone-4-preauth-pipeline && cd capstone-4-preauth-pipeline
python3 -m venv venv && source venv/bin/activate
pip install "anthropic ≥ 0.30.0" pytest
export ANTHROPIC_API_KEY=your-key-here
```

WINDOWS

```
mkdir capstone-4-preauth-pipeline && cd capstone-4-preauth-pipeline
python -m venv venv && venv\Scripts\activate
pip install "anthropic ≥ 0.30.0" pytest
set ANTHROPIC_API_KEY=your-key-here
```

Checkpoint

Run `python -c "import anthropic; print('OK')"`. You should see `OK`. If you see `ModuleNotFoundError`, make sure your virtual environment is activated.

File Structure

Here is every file you will create in this capstone. The build guide walks through them in order.

```
capstone-4-preauth-pipeline/
├─ mock_tools.py           # All agent tools (intake, clinical, decision, communication)
├─ pipeline.py             # Orchestrator: PipelineState, circuit breaker, HITL, all 4 agents
└─ test_pipeline.py        # 6 pytest test cases (happy path, denial, HITL, circuit breaker, invalid input, HIPAA
guardrail)
```

Domain Glossary

Multi-Agent Pipeline

A sequence of specialized agents where each has a focused responsibility and typed input/output schemas. State flows between agents via structured objects, not free text.

Human-in-the-Loop (HITL)

A checkpoint where the pipeline pauses and presents a decision to a human reviewer. The reviewer can approve, deny, modify, or escalate. Pipeline resumes with the reviewer's input.

Circuit Breaker

An automatic safety mechanism that halts processing when consecutive failures exceed a threshold (e.g., 3 failures). Prevents cascading errors. Requires manual reset after investigation.

Pipeline State

A structured object that flows through the pipeline, accumulating outputs from each agent. Each agent reads its predecessors' outputs and writes its own. The state is the single source of truth.

Confidence Score

A 0.0–1.0 value indicating how certain the agent is about its evaluation. Drives routing: high confidence → auto-decide, medium → human review, low → auto-deny.

Input Guardrails

Validation checks applied before an agent processes input. Examples: schema validation, PII detection, duplicate request detection. Prevents garbage-in-garbage-out.

Output Guardrails

Validation checks applied after an agent produces output. Examples: HIPAA compliance check on determination letters, tone validation, required disclosure verification.

Determination Letter

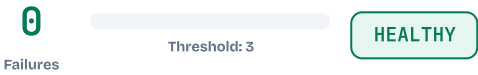
The formal notification sent to providers and members after an authorization decision. Must include the decision, rationale, appeal rights (for denials), and required legal disclosures.

Architecture

4-AGENT PIPELINE — PRE-AUTH PROCESSING

HUMAN-IN-THE-LOOP — MEDIUM CONFIDENCE PATH

CIRCUIT BREAKER — CONSECUTIVE FAILURE PROTECTION



Mock Data Specification

The pipeline state object flows through all 4 agents, accumulating outputs. Each agent reads its predecessors' outputs and writes its own. This typed state schema is the backbone of the multi-agent architecture.

PIPELINE STATE SCHEMA

PIPELINE STATE SCHEMA

```
{
  "request_id": "AR-2024-09821",
  "stage": "intake",          // intake | clinical_criteria | decision | communication | complete | error
  "created_at": "2024-03-10T09:00:00Z",

  "intake_output": {          // Written by Agent 1
    "validated": true,
    "member_verified": true,
    "provider_verified": true,
    "procedure_code": "27447",
    "diagnosis_codes": ["M17.11"],
    "clinical_notes_summary": "Severe right knee OA, KL Grade IV, 8 months PT, NSAIDs, 2 injections, WOMAC 68, BMI 31",
    "missing_fields": [],
    "urgency": "standard"
  },

  "criteria_output": {        // Written by Agent 2
    "policy_id": "POLICY-ORTHO-TKA-2024",
    "criteria_evaluation": [
      {"criterion": "C1", "met": true, "confidence": 0.95, "evidence": "KL Grade IV documented, M17.11 confirmed"},
      {"criterion": "C2", "met": true, "confidence": 0.88, "evidence": "8 months PT, NSAIDs, 2 injections"},
      {"criterion": "C3", "met": true, "confidence": 0.92, "evidence": "WOMAC 68 > 50 threshold"},
      {"criterion": "C4", "met": true, "confidence": 0.98, "evidence": "BMI 31 < 40"}
    ]
  },

  "decision_output": {        // Written by Agent 3
    "overall_confidence": 0.93,
    "determination": "approve",
    "rationale": "All required criteria met with high confidence. In-network provider. Gold PPO plan covers procedure.",
    "human_review_required": false,
    "reviewer_override": null
  },

  "communication_output": {   // Written by Agent 4
    "letter_id": "LTR-2024-09821",
    "letter_type": "approval",
    "sent_via": "portal",
    "sent_at": "2024-03-10T09:08:00Z",
    "hipaa_compliant": true,    // Set by check_hipaa_compliance output guardrail
    "hipaa_issues": [],        // Populated only if guardrail blocked the send
    "blocked": false           // true means the send was halted by the guardrail
  },

  "circuit_breaker": {
    "consecutive_failures": 0,
    "threshold": 3,
    "status": "healthy"        // healthy | tripped
  }
}
```

What Just Happened?

The pipeline state is a structured record that each agent reads and writes to. Agent 1 writes `intake_output`, Agent 2 reads it and writes `criteria_output`, Agent 3 reads both and writes `decision_output`, Agent 4 reads the decision and writes `communication_output`. The `stage` field tracks where in the pipeline the request currently is. The `circuit_breaker` is checked at every transition.

Step-by-Step Build Guide

You will build this pipeline in 3 files: mock tools first, then the pipeline orchestrator (which contains the state, circuit breaker, HITL, and all 4 agents), then the test suite. Each step is independently testable — you will verify each piece works before moving on.

Step 1: Create the Mock Tools (mock_tools.py)

Four agents, each with 2–3 tools. The key insight: each agent has a *focused* set of tools. The Intake Agent cannot evaluate clinical criteria; the Decision Agent cannot draft letters. This enforces separation of concerns.

Create a new file called `mock_tools.py`:

PYTHON (KEY TOOLS) · **NODE.JS (KEY TOOLS)**

PYTHON (KEY TOOLS)

```
import json
import re
from datetime import datetime, timedelta

# =====
# INTAKE AGENT TOOLS
# =====

MEMBER_DB = {
    "MBR-555-1234": {"eligible": True, "plan": "Gold PP0",
                    "effective": "2024-01-01", "termination": None},
}

PROVIDER_DB = {
    "1234567890": {"verified": True, "name": "Dr. Sarah Johnson, MD",
}

# ... (full source in the desktop HTML) ...

issues.append("FORMAT: missing salutation ('Dear ...')")
if "sincerely" not in body_lower and "regards" not in body_lower:
    issues.append("FORMAT: missing sign-off ('Sincerely' or 'Regards')")

return {"compliant": len(issues) == 0, "issues": issues,
        "redacted_text": redacted}
```

NODE.JS (KEY TOOLS)

```
// mock_tools.ts — Key tools for Capstone 4-A (abbreviated)
// Full implementation mirrors the Python version

export function validateAuthRequest(rawRequest): any {
  const required = ["member_id", "provider_npi", "procedure_code", "diagnosis_codes", "clinical_notes"];
  const missing = required.filter(f => !rawRequest[f]);
  if (missing.length) return { validated, missing_fields, normalized_request: null };
  return { validated, missing_fields,
    normalized_request: { procedure_code,
      diagnosis_codes,
      clinical_notes_summary: rawRequest.clinical_notes.slice(0, 200),
      urgency: rawRequest.urgency || "standard" } };
}

# ... (full source in the desktop HTML) ...

if (!body.includes("sincerely") && !body.includes("regards")) {
  issues.push("FORMAT: missing sign-off ('Sincerely' or 'Regards')");
}

return { compliant: 0, issues, redacted_text: redacted };
}
```

Checkpoint

Test the mock tools: `python -c "from mock_tools import validate_auth_request; print(validate_auth_request({}))"`. You should see `{'validated': False, 'missing_fields': ['member_id',`

`'provider_npi', 'procedure_code', 'diagnosis_codes', 'clinical_notes'], 'normalized_request': None}`. If you see an import error, check you created `mock_tools.py` in the project root.

Step 2: Understand the Architecture (No Code — Read Before Building)

Before writing `pipeline.py`, understand the four architectural concepts that are all implemented in that single file:

- **PipelineState dataclass:** The typed contract between agents. Each agent reads its predecessor's output and writes its own. The `stage` field tracks pipeline progress. Defined as a Python `@dataclass` with optional fields for each agent's output.
- **Circuit Breaker:** Three functions — `check_circuit_breaker(state)`, `record_failure(state)`, and `record_success(state)` — track consecutive failures. If 3 failures occur in sequence, the breaker trips, halting the pipeline. A success resets the counter to zero.
- **HITL Reviewer:** When the Decision Agent's confidence falls between 70–90%, the pipeline pauses and presents a CLI review interface. The reviewer sees criteria evaluations, confidence scores, and selects approve/deny/request-info/escalate. The pipeline resumes with the reviewer's override.
- **Agent Runner pattern:** Each of the four agents has its own system prompt, focused tool set, and output schema. **All four** — Intake, Clinical, Decision, and Communication — are dispatched through the same `run_agent` function, which sends messages to Claude with the agent's tools, handles tool calls in a loop until the agent responds with text, and records success/failure on the circuit breaker. Because every agent transition goes through `run_agent`, the circuit breaker is enforced at every step (Agents 1→2→3→4), not just the early ones.
- **Agent 4 output guardrail:** The Communication Agent's tool set includes `check_hipaa_compliance`. The system prompt instructs the LLM to draft → check → send; the orchestrator verifies the guardrail's `compliant` result one more time before sending. If the guardrail fails, the send is blocked and the pipeline records the issues in `communication_output`.

All four concepts live inside `pipeline.py` to keep the project simple. In production, you would split them into separate modules for independent testing and deployment.

Step 3: Build the Pipeline Orchestrator (pipeline.py)

This is the main build step. The orchestrator defines the pipeline state, circuit breaker, HITL reviewer, agent runner, and wires all 4 agents together in sequence. At each transition: check the circuit breaker, log the transition, and validate the output schema before proceeding.

Create a new file called `pipeline.py`:

PYTHON (ORCHESTRATOR)

```

import json
import anthropic
from datetime import datetime
from dataclasses import dataclass, field, asdict
from typing import Optional

client = anthropic.Anthropic()
MODEL = "claude-sonnet-4-6"

# — Pipeline State —————

class PipelineState:
    request_id: str
    stage= "intake"

    # ... (full source in the desktop HTML) ...

    if cmd.lower() == "demo":
        state = run_pipeline(sample)
        print(f"\nFinal state:\n{json.dumps(asdict(state), indent=2, default=str)}")

if __name__ == "__main__":
    main()

```

NODE.JS (ORCHESTRATOR)

```

// pipeline.ts — Multi-Agent Pipeline Orchestrator (Capstone 4-A)
// Abbreviated — mirrors the Python version structure

import Anthropic from "@anthropic-ai/sdk";
import * as readline from "readline";
import {
    validateAuthRequest, verifyMemberEligibility,
    computeDecisionConfidence, finalizeDetermination,
    draftDeterminationLetter, sendNotification, checkHipaaCompliance,
} from "./mock_tools";

const client = new Anthropic();
const MODEL = "claude-sonnet-4-6";

# ... (full source in the desktop HTML) ...

}
ask();
});
ask();
}
main();

```

Checkpoint — Step 3

Run `python pipeline.py` and type `demo`. You should see each agent start and complete in sequence, ending with a determination. If you see `AuthenticationError`, check your `ANTHROPIC_API_KEY`.

Expected Output — Full Pipeline Run

```

[INTAKE] Starting...
[INTAKE] Complete.

[CLINICAL] Starting...
[CLINICAL] Complete.

[DECISION] Starting...
[DECISION] Complete.

```

```
[COMMUNICATION] Starting...
[COMMUNICATION] Complete.

[PIPELINE] Complete! Determination: approve

Final state (truncated):
  "request_id": "AR-2024-09821",
  "stage": "complete",
  "decision_output": { "overall_confidence": 0.93, "determination": "approve", ... },
  "communication_output": { "letter_id": "LTR-DET-AR-2024-09821", "hipaa_compliant": true, ... },
  "circuit_breaker": { "consecutive_failures": 0, "threshold": 3, "status": "healthy" }
```

🎯 What Just Happened?

You built a 4-agent pipeline orchestrator with three production patterns: (1) **typed pipeline state** flowing between agents, (2) **circuit breaker** checking at every transition, (3) **HITL checkpoint** that pauses for human review when confidence is 70–90%. Each agent is independently testable. The orchestrator is the only component that knows the full sequence.

Step 4: Run the Pipeline

Run `python pipeline.py` and type `demo` to process the sample TKA pre-authorization request. Verify you see all four agents execute in sequence and an APPROVE determination.

Step 5: Add Guardrails

Input guardrails go on the Intake Agent: schema validation (are all required fields present?) and PII detection (is there unredacted PHI in fields that shouldn't have it?). Output guardrails go on the Communication Agent: HIPAA compliance check on letter content before send.

The `validate_auth_request` function in `mock_tools.py` already implements the input guardrail (missing-field check). The output guardrail is the `check_hipaa_compliance(letter_text, determination_type)` function you added in Step 1. It runs **three** categories of checks before any letter is sent:

- **PII leakage** — regex match for SSN patterns (`\d{3}-\d{2}-\d{4}`) and full birthdates (`MM/DD/YYYY` or `MM-DD-YYYY`). Any match flags `PII_LEAK` and the offending text is replaced with `[redacted-ssn]` or `[redacted-dob]` in the returned `redacted_text` .
- **Required determination keywords** — approval letters must contain “approved”; denial letters must contain “appeal” instructions; info-request letters must list the items needing submission. A missing keyword flags `MISSING_KEYWORD` .
- **Salutation/sign-off** — the body must start with `Dear` and end with a `Sincerely` or `Regards` sign-off. Missing either flags `FORMAT` .

The Communication Agent's system prompt instructs Claude to `draft_determination_letter` → `check_hipaa_compliance` → (only on `compliant=true`) `send_notification` . The orchestrator double-checks: if `compliant` is false at the end of the agent loop, it sets `state.stage = "error"` , populates `state.communication_output["hipaa_issues"]` , and refuses to send. This belt-and-suspenders pattern ensures a non-compliant draft cannot escape even if the LLM ignores the guardrail.

Checkpoint — Step 5

Two tests to run: (1) Submit a request with `member_id` missing — the Intake Agent rejects it with `validated: False` and `missing_fields: ["member_id"]` ; the pipeline halts at intake. (2) Call `check_hipaa_compliance("Dear Provider, your SSN 123-45-6789 was approved. Sincerely.", "approve")` directly — you should see `compliant=False` , `"PII_LEAK: SSN pattern detected"` in `issues` , and `[redacted-ssn]` in `redacted_text` .

Step 6: Create the Test Suite (test_pipeline.py)

Create `test_pipeline.py` with 6 pytest test cases covering the critical paths: happy-path approval, denial, HITL escalation, circuit breaker trip, invalid input handling, and the HIPAA output guardrail.

PYTHON (TEST_PIPELINE.PY) · NODE.JS (TEST_PIPELINE.TS)

PYTHON (TEST_PIPELINE.PY)

```
import pytest
from unittest.mock import patch
from pipeline import PipelineState, check_circuit_breaker, record_failure, record_success
from mock_tools import (
    validate_auth_request,
    verify_member_eligibility,
    fetch_clinical_policy,
    evaluate_criterion,
    compute_decision_confidence,
    finalize_determination,
    draft_determination_letter,
    send_notification,
    check_hipaa_compliance,
)

# ... (full source in the desktop HTML) ...

# A denial letter missing the word 'appeal' must be flagged
bad_denial = "Dear Provider,\n\nAuthorization is denied.\n\nSincerely,\nTeam"
bd = check_hipaa_compliance(bad_denial, "deny")
assert bd["compliant"] is False
assert any("MISSING_KEYWORD" in i for i in bd["issues"])
```

NODE.JS (TEST_PIPELINE.TS)

```
// test_pipeline.ts - Jest test cases for Capstone 4-A (abbreviated)
// Run: npx jest test_pipeline.ts

import {
    validateAuthRequest, verifyMemberEligibility,
    computeDecisionConfidence, finalizeDetermination,
    draftDeterminationLetter, sendNotification,
} from "../mock_tools";
import { createState, checkCircuitBreaker, recordFailure, recordSuccess } from "../pipeline";

const VALID_REQUEST = {
    request_id: "AR-TEST-001", member_id: "MBR-555-1234",
    provider_npi, procedure_code,
    diagnosis_codes,

    # ... (full source in the desktop HTML) ...

test("invalid input rejected", () => {
    const result = validateAuthRequest({});
    expect(result.validated).toBe(false);
    expect(result.missing_fields).toContain("member_id");
});
```

Run the test suite:

```
pytest test_pipeline.py -v
```

Expected Output

```
test_pipeline.py::test_high_confidence_auto_approve PASSED
test_pipeline.py::test_denial_missing_conservative_treatment PASSED
test_pipeline.py::test_hitl_escalation_borderline_confidence PASSED
test_pipeline.py::test_circuit_breaker_trips PASSED
test_pipeline.py::test_invalid_input_handling PASSED
test_pipeline.py::test_hipaa_guardrail_blocks_ssn_leak PASSED

===== 6 passed in 0.13s =====
```

Run `pytest test_pipeline.py -v`. You should see all 6 tests pass. If any fail, check the troubleshooting section below.

Testing Guide

TYPE	SCENARIO	EXPECTED BEHAVIOR
HAPPY	All criteria met, high confidence (93%)	Full pipeline runs: intake → criteria → auto-approve → letter sent
HAPPY	Medium confidence (72%) triggers HITL	Pipeline pauses at Decision, reviewer approves, pipeline resumes to Communication
HAPPY	Criteria not met, low confidence	Auto-deny with denial letter including appeal rights and rationale
HAPPY	Missing documentation triggers request-info	Pipeline generates info-request letter specifying what's needed
HAPPY	Reviewer overrides deny → approve	Override logged, determination reflects reviewer decision with rationale
EDGE	Intake finds missing required fields	Pipeline stops at intake stage, returns validation errors
EDGE	One criterion has no evidence available	Confidence drops, triggers HITL for that criterion
EDGE	Communication agent fax delivery fails	Agent retries via email channel, logs delivery failure
ADVERSARIAL	Drafted letter contains SSN pattern	HIPAA guardrail flags <code>PII_LEAK</code> , send is blocked, <code>state.stage = error</code>
ADVERSARIAL	3 consecutive pipeline failures	Circuit breaker trips, pipeline halted, operations alerted
ADVERSARIAL	Reviewer takes no action for 24+ hours	System escalates to supervisor review queue

Verify Everything Works

Run the complete pipeline end-to-end with a single command:

UNIX / MACOS - WINDOWS
UNIX / MACOS

```
python pipeline.py <<< "demo" && pytest test_pipeline.py -v
```

WINDOWS

```
echo demo | python pipeline.py && pytest test_pipeline.py -v
```

Expected final output: The pipeline processes the sample pre-auth request (CPT 27447, knee replacement), all 4 criteria are met with high confidence (93%), and the request is auto-approved. The pytest suite then runs all 6 tests (including the HIPAA SSN-leak guardrail test) and reports 6 passed.

Expected Output

```
=====
Pre-Auth Processing Pipeline – Capstone 4-A
Type 'demo' for sample request, or 'quit' to exit.
=====

[INTAKE] Starting...
[INTAKE] Complete.

[CLINICAL] Starting...
[CLINICAL] Complete.

[DECISION] Starting...
[DECISION] Complete.

[COMMUNICATION] Starting...
[COMMUNICATION] Complete.

[PIPELINE] Complete! Determination: approve
```

```
--- pytest test_pipeline.py -v ---
test_pipeline.py::test_high_confidence_auto_approve PASSED
test_pipeline.py::test_denial_missing_conservative_treatment PASSED
test_pipeline.py::test_hitl_escalation_borderline_confidence PASSED
test_pipeline.py::test_circuit_breaker_trips PASSED
test_pipeline.py::test_invalid_input_handling PASSED
test_pipeline.py::test_hipaa_guardrail_blocks_ssn_leak PASSED

===== 6 passed in 0.13s =====
```

CONGRATULATIONS!

You have built a complete multi-agent pre-authorization pipeline with four specialized agents, a circuit breaker that halts on cascading failures, and human-in-the-loop review for medium-confidence clinical decisions. This is the same architecture used in production healthcare systems — the only differences in a real deployment would be connecting to actual payer policy APIs, authenticated HITL reviewer dashboards, and encrypted PHI handling under a BAA.

Troubleshooting

Common Errors

IMPORT ERRORS

ModuleNotFoundError: No module named 'anthropic' — Your virtual environment is not activated. Run `source venv/bin/activate` (Unix) or `venv\Scripts\activate` (Windows).

ModuleNotFoundError: No module named 'mock_tools' — You are running the script from the wrong directory. Make sure you are inside `capstone-4-preauth-pipeline/`.

ImportError: cannot import name 'PipelineState' from 'pipeline' — The `pipeline.py` file is missing or has a syntax error. Complete Steps 1–3 before running tests.

API ERRORS

AuthenticationError: 401 — Your `ANTHROPIC_API_KEY` is not set or is invalid. Run `echo $ANTHROPIC_API_KEY` (Unix) or `echo %ANTHROPIC_API_KEY%` (Windows) to verify.

RateLimitError: 429 — You are sending too many requests. Wait a few seconds and retry. Each pipeline run makes 4 API calls (one per agent), so budget accordingly.

PIPELINE LOGIC ERRORS

HITL never triggers — Check that your test request produces a confidence score between 0.70 and 0.90. Reduce the number of keyword matches in clinical notes to lower confidence (e.g., remove “KL Grade IV” to drop C1 confidence).

Circuit breaker does not trip — Ensure your test submits 3 requests that trigger agent failures (e.g., invalid JSON, missing member IDs). The breaker counts *consecutive* failures — a success in between resets the counter.

All criteria show confidence 0.0 — The mock evaluation function matches keywords case-insensitively. Check that your clinical notes contain the expected keywords: “M17.11”, “PT”, “WOMAC”, “BMI”.

WINDOWS-SPECIFIC ISSUES

'python3' is not recognized — On Windows, use `python` instead of `python3`. Check your version with `python --version`.

'source' is not recognized — On Windows Command Prompt, use `venv\Scripts\activate` (no `source`). On PowerShell, use `.\venv\Scripts\Activate.ps1`.

HIPAA Compliance Notes

⚠️ HIPAA — MULTI-AGENT PIPELINE AND PHI

A multi-agent pipeline multiplies HIPAA surface area: every agent, every inter-agent state transfer, every tool call, and every log entry may contain PHI. Production requirements:

- **Inter-agent state encryption:** Pipeline state objects contain member IDs, diagnosis codes, and clinical notes. Encrypt at rest and in transit between agents, even within the same process.
- **Per-agent audit trails:** Each agent must log every tool call with timestamp, input/output (redacted for minimum necessary), and agent identity. These logs are PHI access records.
- **HITL reviewer authentication:** The human reviewer must be authenticated and authorized. Log reviewer identity, decision, and rationale as part of the medical record.
- **Determination letter compliance:** Denial letters must include appeal rights, clinical rationale, and state-mandated disclosures. The Communication Agent's output guardrail must verify these are present.
- **Circuit breaker notifications:** When the circuit breaker trips, the alert must NOT include PHI. Alert format: "Pipeline halted at [stage] due to [error count] failures. Request IDs: [redacted]."
- **BAA coverage:** Every Claude API call in the pipeline sends PHI. All 4 agents' API calls must be covered by a BAA with Anthropic.

💰 COST: 4 AGENTS = 4X API CALLS

Each pipeline run makes 4 separate Claude API calls (one per agent), plus tool call round-trips within each agent. A typical 4-agent pipeline consumes 8,000–15,000 tokens total. At Sonnet pricing (~\$3/M input, \$15/M output), budget ~\$0.05–\$0.10 per authorization. For 500 auths/day, that's \$25–\$50/day in API cost. The HITL path adds latency but not token cost (human review is offline).